d00edeb2-121d-4446-ab95-fb3fe0b93798

NTUA - Τεχνολογίες Υπη…

Κοινή χρήση

NTUA-saasPlug-main.zip
Συμπιεσμένο αρχείο

We are continuing the NTUA ECE course project **“Technologies of Software Services / Τεχνολογίες Υπηρεσιών Λογισμικού”**. The project is called **saasPlug**.

The discussion can be in Greek, but **all repository files, documentation, code, comments, README files, Codex prompts, and AI logs must be in English**.

I will attach the current repository zip. Your first task is to inspect it deeply and confirm that you understand the full

Εμφάνιση περισσότερων

Σκέφτηκε για 4m 29s

Επιβεβαίωση ελέγχου repository zip

Επιθεώρησα το `NTUA-saasPlug-main.zip`. Η εικόνα είναι συνεπής με το σημείο που περιέγραψες: το repo βρίσκεται **μετά το Prompt 00B** και **πριν το Prompt 01**.

Συμπέρασμα

Έλεγχος	Κατάσταση
Repo έτοιμο για Prompt 01	**Ναι**
AI logging structure	**Σωστό και πλέον οργανωμένο**
`AGENTS.md` και `docs/specs/SPECIFICATIONS.md`	**Παρόντα, συνεκτικά, canonical**
Blockers πριν το Prompt 01	**Κανένας ουσιαστικός**
Docker/GitHub setup	**Φαίνεται χειρισμένο βάσει logs, όχι ανεξάρτητα επαληθεύσιμο από zip**
Zip current after Prompt 00B	**Ναι**

Σημεία που επιβεβαιώθηκαν

- Υπάρχουν και είναι αναγνώσιμα:
  - `AGENTS.md`
  - `docs/specs/SPECIFICATIONS.md`
  - `docs/ai/prompt-log.md`
  - `docs/ai/prompts/prompt_00b_logging_and_github_setup.md`
  - `docs/ai/prompts/prompt_01_repo_foundation.md`
  - `docs/ai/codex-runs/000-audit-only.md`
  - `docs/ai/codex-runs/00B-logging-and-github-setup.md`
  - `docs/ai/codex-runs/_template.md`
- Το `prompt-log.md` έχει:
  - Run `000` ως completed.
  - Run `00B` ως `completed_with_warnings`.
  - Run `001` ως planned.
  - Άρα το Prompt 01 είναι πράγματι το επόμενο βήμα.
- Τα provider OpenAPI YAMLs υπάρχουν και είναι έγκυρα ως OpenAPI 3.1:
  - `redPlug.yaml`: `/points`, `/point/{pointid}`, `/reserve/{pointid}`, `/reserve/{pointid}/{minutes}`
  - `greenPlug.yaml`: `/chargingPoints`, `/chargingPoints/{pointid}`, `/chargingPoints/{pointid}/reservations`
  - `bluePlug.yaml`: `/locations`, `/location/{pointid}/status`, `/location/{pointid}/hold`
- Τα assignment/provider artifacts υπάρχουν στη σωστή περιοχή:
  - SRS `.docx`
  - εκφώνηση PDF
  - provider API PDF
  - VPP
  - PlantUML exports
  - YAMLs
  - Postman collections

Warnings / μη-blocking θέματα

- Δεν υπάρχουν ακόμα `apps/`, `services/`, `packages/`, `deploy/`, `scripts/`. Αυτό είναι **αναμενόμενο**, γιατί είναι ακριβώς scope του Prompt 01.
- Δεν υπάρχει ακόμα `.env.example`. Αυτό επίσης είναι **Prompt 01 scope**.
- Το root `README.md` είναι πολύ minimal και το `docs/README.md` είναι κενό. Το Prompt 01 πρέπει να τα συμπληρώσει.

- Το `docs/assignment/README.md` και το `AGENTS.md` αναφέρουν `Project Notes from Lectures.docx`, αλλά δεν το βρήκα στο zip. Δεν μπλοκάρει το Prompt 01, αλλά να το έχουμε ως warning για επόμενα prompts.
- Το zip δεν περιέχει `.git/`, άρα δεν μπορώ να επαληθεύσω απευθείας το local `origin` remote ή το live GitHub Project. Το `00B` run log όμως καταγράφει ότι το GitHub setup ολοκληρώθηκε.
- Υπάρχουν ιστορικές αναφορές σε `saasCharge` μέσα στο SRS/intermediate diagrams. Δεν είναι blocker, γιατί τα intermediate/historical artifacts δεν είναι canonical. Τα final artifacts πρέπει να ακολουθούν `saasPlug`.

Απόφαση

Προχώρα με **Prompt 01 - Repository Foundation**.

Επεξεργασία

Read and execute docs/ai/prompts/prompt_01_repo_foundation.md.

Follow AGENTS.md and docs/specs/SPECIFICATIONS.md.

Use docs/ai/codex-runs/_template.md for the run report.

Update docs/ai/codex-runs/001-repo-foundation.md and docs/ai/prompt-log.md.

Record both the exact invocation prompt and the canonical task prompt file/content where practical.

Do not implement business logic, services, frontend screens, Docker runtime, or database schemas beyond the foundation scope of Prompt 01.

NTUA-saasPlug-main (1…
Συμπιεσμένο αρχείο

Αυτό είναι το ανανεωμένο .zip του repo και η έξοδος του Codex:

Implemented Prompt 01 foundation only.

Created files
- `.env.example`

- README scaffolds under `apps/`, `services/`, `packages/`, `deploy/`, `scripts/`
- [001 run report](/home/peppasmich/NTUA-saasPlug/docs/ai/codex-runs/001-repo-foundation.md)

Εμφάνιση περισσότερων

NTUA-saasPlug-main (2…
Συμπιεσμένο αρχείο

Αυτό είναι το ανανεωμένο .zip του repo και η έξοδος του Codex:

COMPLETED

Υλοποιήθηκε το Prompt 02 baseline: [docker-compose.yml](/home/peppasmich/NTUA-saasPlug/docker-compose.yml), Nginx gateway, minimal Vite frontend placeholder, health-only FastAPI scaffolds για τα 10 canonical services, PostgreSQL ανά stateful service,

Εμφάνιση περισσότερων

Έλεγχος Prompt 02

Με βάση το νέο ZIP και την έξοδο του Codex, το Prompt 02 φαίνεται **ολοκληρωμένο σωστά**.

Σημείο	Κατάσταση
`docker-compose.yml`	OK
Nginx gateway	OK
Frontend placeholder	OK
10 FastAPI service scaffolds	OK
10 PostgreSQL containers	OK
RabbitMQ	OK
Gateway health routes	OK
`docs/ai/codex-runs/002-docker-gateway.md`	OK
`docs/ai/prompt-log.md`	OK, Run 002 marked completed
Business logic accidentally added	Δεν φαίνεται
Blocker πριν το Prompt 03	Κανένας ουσιαστικός

Η προειδοποίηση ότι το GitHub issue #2 παραμένει open είναι **μη-blocking**, αφού το prompt απαγόρευε GitHub mutations.

Επόμενο prompt

Το επόμενο canonical prompt είναι:

```
docs/ai/prompts/prompt_03_auth_service.md
```

Το αρχείο που πρέπει να βάλεις στο repo είναι εδώ:

Download prompt_03_auth_service.md

Αποθήκευσέ το ακριβώς στο:

```
docs/ai/prompts/prompt_03_auth_service.md
```

Invocation prompt για Codex

Επεξεργασία

Read and execute docs/ai/prompts/prompt_03_auth_service.md.

Follow AGENTS.md and docs/specs/SPECIFICATIONS.md.

Use docs/ai/codex-runs/_template.md for the run report.

Update docs/ai/codex-runs/003-auth-service.md and docs/ai/prompt-log.md.

Record both the exact invocation prompt and the canonical task prompt file/content where practical.

Implement only the auth-service scope defined in Prompt 03: local auth persistence, email/password login, JWT access tokens, refresh tokens, seeded demo users, role/provider scoping helpers, safe Google OAuth hook endpoints, tests, documentation, and validation.

Do not implement frontend login screens, provider-registry business logic, provider adapters, catalog sync/search, status cache logic, reservation flow, analytics, billing, RabbitMQ business consumers, real secrets, GitHub issue/project mutations, or fallback Markdown project-management artifacts.

Θα το δέσω με το υπάρχον prompt sequence του repo, όχι από μνήμη μόνο, ώστε να μην αλλάξουμε τη σειρά αυθαίρετα.

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 10s

The planned remaining prompts are:

Prompt	File	Purpose
12	`docs/ai/prompts/prompt_12_frontend_ev_flows.md`	EV user frontend: login, map/search, point details, status lookup, reservation flow
13	`docs/ai/prompts/prompt_13_frontend_admin_dashboards.md`	Provider/admin and operator dashboards: analytics, billing, pricing, sync controls
14	`docs/ai/prompts/prompt_14_docs_uml_finalization.md`	Final documentation, PlantUML diagrams, README completion, final verification

This matches the repo's `docs/ai/prompts/README.md` planned sequence.

README

Practical interpretation

Prompt 12 - EV frontend flows

This should connect the existing backend into the first usable user-facing flow:

```
login -> map/search -> point details -> status -> reserve -> my reservations
```

Expected scope:

- React + TypeScript + Vite frontend implementation.
- Auth/login state using existing `auth-service`.
- EV user pages/components.
- Catalog map/list/search through gateway only.
- Point details from `catalog-service`.
- Status lookup through `status-service`.

- Reservation creation/history through `reservation-service`.
- No provider/operator dashboards yet.

Prompt 13 - Admin/operator frontend dashboards

This should implement role-specific B2B/operator screens:

- Provider dashboard:
  - own provider analytics,
  - usage logs/export,
  - own invoices,
  - mock invoice payment,
  - provider settings/config visibility where appropriate.
- Operator dashboard:
  - global analytics,
  - provider comparison,
  - pricing-rule management,
  - invoice issuance,
  - sync/import controls,
  - billing overview.

This should consume existing backend APIs, not create new architecture.

Prompt 14 - Finalization

This should be the final cleanup and delivery run:

- Final README pass.
- Final PlantUML diagrams under:

```
docs/diagrams/final/
```

- Architecture documentation alignment.
- Verification matrix.
- End-to-end smoke documentation.
- AI logs completeness check.
- Final course-deliverable polish.

So after Prompt 11, the project shifts from backend/service implementation into **frontend integration**, then **final documentation/verification**.

Great, now, that I will paste the new output of Codex and tell you to inspect the repo, I want you to make sure we are ready to move on to the next prompt. For the frontend creation (all prompts that refer to it), you may find some of these skills useful. I want Codex to create a professional and polished result:

Use the relevant installed frontend/design skills and plugins when useful:

@Figma

Εμφάνιση περισσότερων

Θα ελέγξω πρώτα το Prompt 11 από τα canonical logs και μετά το τρέχον frontend scaffold, ώστε το Prompt 12 να είναι ακριβές και να ενσωματώνει τις design/frontend skill οδηγίες χωρίς deployment scope.

Έγινε λήψη της απάντησης ης εφαρμογής

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 2m 17s

Έλεγχος μετά το Prompt 11

Το repo φαίνεται **εντάξει και έτοιμο για Prompt 12**.

Έλεγχος	Κατάσταση
`docs/ai/prompt-log.md`	OK - το run `011` είναι `completed` prompt-log
Prompt 11 run report	OK - τελικό status `COMPLETED` 011-billing-service
Billing-service scope	OK - pricing, charges, invoices, payments, event ingestion, outbox publishing υλοποιήθηκαν 011-billing-service
Tests/checks	OK - billing tests, event package tests, compose, full stack, gateway/billing health πέρασαν 011-billing-service
Issue #11	OK - closed/completed Prompt 11 - Billing service

Έλεγχος	Κατάσταση
Issue #12	Open, όπως αναμένεται για το επόμενο prompt Prompt 12 - EV frontend flows
Frontend	Ακόμα placeholder, άρα σωστό επόμενο βήμα είναι Prompt 12 README
Frontend stack	React + TypeScript + Vite, currently minimal dependencies only package

Δεν υπάρχει blocker. Το Prompt 12 πρέπει να αντικαταστήσει το placeholder UI με polished EV-user frontend flows, χωρίς provider/operator dashboards ακόμα.

Νέο prompt file

Αποθήκευσέ το στο repo εδώ:

```
docs/ai/prompts/prompt_12_frontend_ev_flows.md
```

Download prompt_12_frontend_ev_flows.md

Invocation prompt για Codex

```
Read and execute docs/ai/prompts/prompt_12_frontend_ev_flows.md.

Follow AGENTS.md and docs/specs/SPECIFICATIONS.md.

Use docs/ai/codex-runs/_template.md for the run report.

Update docs/ai/codex-runs/012-frontend-ev-flows.md and docs/ai/prompt-log.md.

Record both the exact invocation prompt and the canonical task prompt file/content

Implement only the EV frontend scope defined in Prompt 12: login/register/logout, s

Use the relevant installed frontend/design skills and plugins when useful: @Figma,

Do not use Vercel, do not deploy, and do not perform hosting setup. Work only on th

After all Prompt 12 acceptance criteria pass, inspect, comment on, and close GitHub

If GitHub issue/project mutations fail, report the failure clearly in the Codex fir

Do not implement provider admin dashboards, operator dashboards, analytics dashboar
```

Επιθεώρησε εις βάθος το ανανεωμένο repo μου online με το connector που έχεις διαθέσιμο και αυτή είναι η έξοδος του Codex:

COMPLETED_WITH_WARNINGS

Implemented Prompt 12 EV frontend flows in the local React/Vite app: auth/session handling, login/register/logout, EV-only map/search/list, point details, status lookup, reservation creation, own reservation history, profile, responsive styling, and frontend tests. No backend

Εμφάνιση περισσότερων

Θα ελέγξω το Prompt 12 run report, το prompt-log, το issue #12/#13 και βασικά frontend αρχεία. Μετά θα δώσω Prompt 13 με ρητές οδηγίες για polished provider/operator dashboards και χρήση των σχετικών frontend/design skills χωρίς deployment.

Έγινε λήψη της απάντησης ης εφαρμογής

Το Prompt 12 είναι αποδεκτό με μη-blocking warnings: δεν έγινε browser/Figma QA λόγω εργαλείων/απουσίας Figma file, όχι λόγω λειτουργικής αποτυχίας. Θα ελέγξω τώρα την τρέχουσα routing/API δομή για να μην ζητήσει το Prompt 13 ασύμβατες frontend αλλαγές.

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 1m 30s

Έλεγχος μετά το Prompt 12

Το repo φαίνεται **εντάξει και έτοιμο για Prompt 13**.

Έλεγχος	Κατάσταση
`docs/ai/prompt-log.md`	OK - το run `012` είναι `completed_with_warnings` prompt-log
Prompt 12 run report	OK - EV frontend flows υλοποιήθηκαν και καταγράφηκαν 012-frontend-ev-flows
Tests/checks	OK - `npm run test`, `npm run build`, Compose, gateway/frontend/health/API smokes πέρασαν 012-frontend-ev-flows

Έλεγχος	Κατάσταση
Warnings	Μη-blocking - browser/Figma QA δεν έγινε λόγω tool/Figma input limitations, όχι λόγω build/runtime αποτυχίας 012-frontend-ev-flows
Issue #12	OK - closed/completed Prompt 12 - EV frontend flows
Issue #13	Open, όπως αναμένεται για το επόμενο prompt Prompt 13 - Provider and operat…
Frontend routing	Υπάρχουν μόνο EV/auth/profile routes προς το παρόν, άρα Prompt 13 είναι σωστό επόμενο βήμα App
API safety	Ο frontend API client επιβάλλει gateway-only `/api/...` paths api

Δεν βλέπω blocker. Το Prompt 13 πρέπει να υλοποιήσει **μόνο provider-admin και operator frontend dashboards**, όχι backend αλλαγές.

Νέο prompt file

Αποθήκευσέ το στο repo εδώ:

```
docs/ai/prompts/prompt_13_provider_operator_dashboards.md
```

Download prompt_13_provider_operator_dashboards.md

Invocation prompt για Codex

```
Read and execute docs/ai/prompts/prompt_13_provider_operator_dashboards.md.

Follow AGENTS.md and docs/specs/SPECIFICATIONS.md.

Use docs/ai/codex-runs/_template.md for the run report.

Update docs/ai/codex-runs/013-provider-operator-dashboards.md and docs/ai/prompt-lo

Record both the exact invocation prompt and the canonical task prompt file/content

Implement only the provider-admin and operator frontend dashboard scope defined in

Use the relevant installed frontend/design skills and plugins when useful: @Figma,

Do not use Vercel, do not deploy, and do not perform hosting setup. Work only on th
```

```
After all Prompt 13 acceptance criteria pass, inspect, comment on, and close GitHub

If GitHub issue/project mutations fail, report the failure clearly in the Codex fir

Do not implement backend services, backend database schemas, new backend endpoints,
```

Επιθεώρησε εις βάθος το ανανεωμένο repo μου online με το connector που έχεις διαθέσιμο και αυτή είναι η έξοδος του Codex:

COMPLETED_WITH_WARNINGS

Implemented Prompt 13 provider-admin and operator frontend dashboards in the existing React/Vite app. Added role-aware redirects/routes, provider pages, operator pages, shared dashboard UI, frontend DTOs/formatters, CSV export support, responsive styling, and routing tests.

Εμφάνιση περισσότερων

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 2m 4s

Έλεγχος μετά το Prompt 13

Το repo φαίνεται **λειτουργικά έτοιμο για Prompt 14**, με μία σημαντική προϋπόθεση για το τελικό verification: **άνοιξε πρώτα το Docker Desktop Linux engine**, ώστε το final prompt να μπορέσει να τρέξει full-stack smoke tests.

Έλεγχος	Κατάσταση
`docs/ai/prompt-log.md`	OK - το run `013` είναι `completed_with_warnings` prompt-log
Prompt 13 implementation	OK - provider/operator dashboards υλοποιήθηκαν μέσα στο υπάρχον React/Vite app 013-provider-operator-dashboards
Frontend tests/build	OK - `npm run test` και `npm run build` πέρασαν 013-provider-operator-dashboards
Forbidden frontend calls	OK - δεν βρέθηκαν direct calls σε internal ports/provider APIs/RabbitMQ 013-provider-operator-dashboards

Έλεγχος	Κατάσταση
Runtime smoke	Δεν ολοκληρώθηκε - Docker daemon unavailable 013-provider-operator-dashboards
Issue #13	OK - closed/completed Prompt 13 - Provider and operat…
Issue #14	Open, όπως αναμένεται για το τελικό prompt Prompt 14 - Final docs and UML …

Δεν βλέπω source-code blocker. Το μόνο πρακτικό warning είναι ότι το Prompt 14 πρέπει να ξανατρέξει Docker/gateway validation όταν το Docker Desktop είναι διαθέσιμο.

Τι θα κάνει το Prompt 14

Το Prompt 14 είναι **τελικό documentation + UML + verification pass**. Δεν θα προσθέσει νέα features.

Θα κάνει:

1. **Final documentation**
   - root `README.md` final pass,
   - `docs/architecture-summary.md`,
   - `docs/final-verification.md`,
   - `docs/submission-checklist.md`,
   - README coverage/cross-links.
2. **Final PlantUML diagrams**
   - use case,
   - component,
   - deployment,
   - data model,
   - activity diagrams,
   - sequence diagrams,

     όλα κάτω από:

     ```
     docs/diagrams/final/
     ```
3. **Final verification**
   - frontend tests/build,
   - shared events tests,
   - Docker Compose config,
   - full-stack health checks if Docker is available,

- authenticated smoke tests if Docker is available,
- `.env`/secret checks,
- forbidden direct-call checks.

4. **AI/GitHub closure**
   - creates `docs/ai/codex-runs/014-final-docs-uml.md`,
   - updates `docs/ai/prompt-log.md`,
   - closes issue `#14` if acceptance criteria pass.

Νέο prompt file

Αποθήκευσέ το στο repo εδώ:

```
docs/ai/prompts/prompt_14_final_docs_uml.md
```

Download prompt_14_final_docs_uml.md

Invocation prompt για Codex

```
Read and execute docs/ai/prompts/prompt_14_final_docs_uml.md.

Follow AGENTS.md and docs/specs/SPECIFICATIONS.md.

Use docs/ai/codex-runs/_template.md for the run report.

Update docs/ai/codex-runs/014-final-docs-uml.md and docs/ai/prompt-log.md.

Record both the exact invocation prompt and the canonical task prompt file/content

Implement only the final documentation, final PlantUML UML diagrams, README complet

Use the relevant installed documentation/frontend/design/audit skills and plugins v

Use Figma only if a usable Figma file/node/source is actually available. Do not fak

Do not use Vercel, do not deploy, and do not perform hosting setup. Work only on th

Before finalizing, rerun the available Prompt 13/whole-system verification checks.

After all Prompt 14 acceptance criteria pass, inspect, comment on, and close GitHub

If GitHub issue/project mutations fail, report the failure clearly in the Codex fir

Do not implement backend business logic, frontend features, backend database schema
```

Codex finished with this output:

COMPLETED_WITH_WARNINGS.

Final documentation is complete: root/service/deploy/docs READMEs were finalized, `docs/architecture-summary.md`, `docs/final-verification.md`, and `docs/submission-checklist.md` were added, and the full final PlantUML set was created under `docs/diagrams/final/`. AI logging was updated in

Εμφάνιση περισσότερων

Έγινε λήψη της απάντησης ης εφαρμογής

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 36s

Κατάσταση τώρα

Είμαστε σε καλό σημείο για νέα συνομιλία.

Επιβεβαίωσα από το online repo ότι το `docs/ai/prompt-log.md` έχει πλέον runs `000` έως `014`, με το `014` ως `completed_with_warnings`.

prompt-log

Το Prompt 14 δημιούργησε τα τελικά docs, το verification report, το submission checklist και 13 final PlantUML αρχεία κάτω από `docs/diagrams/final/`. 014-final-docs-uml

Το τελικό verification πέρασε για frontend tests/build, shared event tests με workaround, Docker Compose config/startup/health, gateway health, authenticated EV/provider/operator smoke, README coverage, forbidden frontend source checks και obsolete-name checks.

014-final-docs-uml

Τα warnings είναι μόνο για optional/local tooling: Browser Use localhost automation, Figma source, PlantUML rendering και το γνωστό Python 3.14/pytest capture θέμα.
014-final-docs-uml Το GitHub issue #14 είναι επίσης closed/completed.
Prompt 14 - Final docs and UML …

Παρακάτω είναι το prompt για τη νέα συνομιλία.

```
We are continuing the NTUA ECE course project "Technologies of Software Services /

The discussion can be in Greek, but all repository files, documentation, code, comm

I will provide access to the GitHub repository through the available connector. You
```

Repository target:

ntua-el21026/NTUA-saasPlug

Project name:

saasPlug

Course context:

This is an NTUA ECE semester project for "Technologies of Software Services / Τεχνο

The project integrates three hypothetical providers:

- redPlug
- greenPlug
- bluePlug

Each provider has its own OpenAPI YAML specification under the assignment sources.

Important language/content rule:

- We may discuss in Greek.
- All repository content must remain in English.
- Do not propose Greek repository filenames, comments, docs, or code.
- Do not suggest deployment unless explicitly asked.
- Do not use Vercel or any hosting/deployment workflow.

Canonical repository files:

You must treat these as authoritative:

- AGENTS.md
- docs/specs/SPECIFICATIONS.md
- docs/ai/prompt-log.md
- docs/ai/codex-runs/
- docs/ai/prompts/README.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/architecture-summary.md
- docs/diagrams/final/

If anything else conflicts with AGENTS.md or docs/specs/SPECIFICATIONS.md, follow A

Accepted architecture summary:

1. One role-based frontend web app:
   - React + TypeScript + Vite.
   - Located under apps/frontend.
   - Browser calls only gateway routes under /api/...
   - No browser calls to internal service ports.

```
   - No browser calls to external provider APIs.
   - No browser RabbitMQ access.

2. Backend:
   - Python FastAPI microservices.
   - Each service owns its data and its own PostgreSQL container if stateful.

3. Gateway:
   - Nginx reverse proxy/API gateway.
   - Frontend calls only gateway paths.
   - No direct browser access to internal services or provider APIs.

4. Runtime:
   - Docker Compose.

5. Databases:
   - PostgreSQL only.
   - Separate PostgreSQL container per stateful service.
   - No MongoDB.

6. Messaging:
   - RabbitMQ topic exchange.
   - RabbitMQ is used for background/event flows:
     - provider sync,
     - catalog aggregation/import events,
     - analytics aggregation,
     - billing propagation.
   - RabbitMQ is not used for:
     - login,
     - map search,
     - point details,
     - status lookup,
     - the live reservation provider call,
     - direct frontend actions.

7. No dedicated orchestrator service:
   - Use local orchestration inside responsible services.
   - Use event choreography for asynchronous propagation.
   - No saga/distributed transaction implementation for MVP.

Canonical service set:

- auth-service
- provider-registry-service
- red-provider-service
- green-provider-service
- blue-provider-service
- catalog-service
- status-service
- reservation-service
- analytics-service
- billing-service
```

Data/service ownership:

- auth-service:
  - users,
  - password hashes,
  - JWTs,
  - refresh tokens,
  - roles,
  - provider-admin scope,
  - Google OAuth hooks.

- provider-registry-service:
  - provider metadata,
  - provider activation status,
  - provider API configuration metadata,
  - safe config audit logs.

- red-provider-service / green-provider-service / blue-provider-service:
  - provider-local snapshots,
  - raw payloads,
  - sync runs,
  - provider-specific normalization,
  - provider status/reservation adapter calls.

- catalog-service:
  - aggregated normalized charging-point catalog,
  - viewport/bounds map loading,
  - cached point details,
  - import runs,
  - freshness metadata.

- status-service:
  - 5-minute status cache,
  - status check logs,
  - fallback to last known status on timeout/error.

- reservation-service:
  - reservation truth,
  - synchronous provider reservation calls through provider services,
  - successful/failed/unknown attempts,
  - reservation event outbox.

- analytics-service:
  - idempotent raw usage_events,
  - aggregate/cache tables,
  - provider/operator analytics,
  - CSV export,
  - no direct SQL access to billing/reservation/catalog DBs.

- billing-service:
  - pricing rules,

```
  - charges,
  - invoices,
  - invoice lines,
  - mock full-invoice payments,
  - billing event ingestion/outbox,
  - financial source of truth.

- packages/events:
  - shared event envelope,
  - routing-key constants,
  - RabbitMQ topology declarations,
  - publisher/consumer helpers,
  - idempotency helpers.

Frontend scope already implemented:

Prompt 12 implemented EV flows:

- login,
- register,
- logout,
- session handling,
- EV map/search/list,
- point details,
- status lookup,
- reservation creation,
- own reservation history,
- profile,
- responsive UI,
- frontend tests/build validation.

Prompt 13 implemented provider-admin/operator dashboards:

- role-aware redirects/routes,
- provider dashboard,
- provider points,
- provider analytics,
- provider invoices/payment,
- provider export/settings,
- operator dashboard,
- provider management,
- operator analytics,
- catalog/sync controls,
- invoice issuance,
- pricing controls,
- system overview,
- responsive dashboard UI,
- routing tests.

Implementation history:

The project was implemented through staged Codex prompts. Inspect docs/ai/prompt-lo
```

- 000: completed
- 00B: completed_with_warnings
- 001: completed
- 002: completed
- 003: completed
- 004: completed
- 005: completed
- 006: completed_with_warnings
- 007: completed_with_warnings
- 008: completed
- 009: completed
- 010: completed
- 011: completed
- 012: completed_with_warnings
- 013: completed_with_warnings
- 014: completed_with_warnings

The warnings so far have been acceptable and mostly environmental/tooling-related:
- provider live endpoint smoke affected by placeholder credentials,
- Browser Use localhost automation unavailable,
- no Figma source provided,
- PlantUML CLI not installed,
- Python 3.14/pytest capture issue that passed with -s,
- one temporary Docker daemon availability issue in Prompt 13 that was later resolv

Prompt 14 finalization:

Codex finished Prompt 14 with COMPLETED_WITH_WARNINGS.

It added/finalized:

- README.md
- docs/architecture-summary.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/diagrams/final/
- docs/ai/codex-runs/014-final-docs-uml.md
- docs/ai/prompt-log.md

It also updated final README coverage and documentation cross-links.

Final diagrams expected under docs/diagrams/final/ include:

- use-case.puml
- component.puml
- deployment.puml
- data-model.puml
- activity-search-reservation.puml
- activity-provider-sync.puml
- activity-invoice-payment.puml
- sequence-login.puml

- sequence-search-map.puml
- sequence-status-lookup.puml
- sequence-reservation.puml
- sequence-provider-sync-catalog-import.puml
- sequence-analytics-billing-events.puml

Prompt 14 validation passed:

- frontend tests,
- frontend build,
- shared events tests with -s workaround,
- docker compose config,
- docker compose up -d --build,
- docker compose ps all healthy,
- gateway health,
- all public service health routes,
- authenticated EV smoke,
- authenticated provider-admin smoke,
- authenticated operator smoke,
- docker compose down -v,
- no remaining containers,
- git diff --check,
- no .env,
- forbidden frontend/source checks,
- obsolete diagram-name checks,
- README coverage check.

Prompt 14 warnings:

- pytest without -s hit local Python 3.14 capture issue, but with -s all shared eve
- Browser Use localhost automation was not exposed by available tools.
- No usable Figma source was provided.
- plantuml CLI is not installed, so PlantUML diagram rendering was not locally vali
- docs/ai/prompts/prompt_14_final_docs_uml.md was already untracked and left unmod

Your task in this new ChatGPT conversation:

Do not implement anything yourself.

Use the GitHub connector to deeply inspect the actual repository. Do not rely only

Inspect at least:

Root and governance:
- AGENTS.md
- README.md
- .gitignore
- .env.example
- docker-compose.yml
- deploy/nginx/default.conf

Specs and final docs:

- docs/specs/SPECIFICATIONS.md
- docs/specs/README.md
- docs/architecture-summary.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/README.md

AI logs:
- docs/ai/prompt-log.md
- docs/ai/codex-runs/000-audit-only.md
- docs/ai/codex-runs/00B-logging-and-github-setup.md
- docs/ai/codex-runs/001-repo-foundation.md
- docs/ai/codex-runs/002-docker-gateway.md
- docs/ai/codex-runs/003-auth-service.md
- docs/ai/codex-runs/004-provider-registry-service.md
- docs/ai/codex-runs/005-provider-services.md
- docs/ai/codex-runs/006-catalog-service.md
- docs/ai/codex-runs/007-status-service.md
- docs/ai/codex-runs/008-reservation-service.md
- docs/ai/codex-runs/009-rabbitmq-event-infrastructure.md
- docs/ai/codex-runs/010-analytics-service.md
- docs/ai/codex-runs/011-billing-service.md
- docs/ai/codex-runs/012-frontend-ev-flows.md
- docs/ai/codex-runs/013-provider-operator-dashboards.md
- docs/ai/codex-runs/014-final-docs-uml.md

Frontend:
- apps/frontend/README.md
- apps/frontend/package.json
- apps/frontend/package-lock.json
- apps/frontend/src/App.tsx
- apps/frontend/src/components/
- apps/frontend/src/features/
- apps/frontend/src/lib/
- apps/frontend/src/styles/
- apps/frontend/src/test/

Shared package:
- packages/events/README.md
- packages/events/saasplug_events/
- packages/events/tests/

Services:
- services/README.md
- services/auth-service/
- services/provider-registry-service/
- services/red-provider-service/
- services/green-provider-service/
- services/blue-provider-service/
- services/catalog-service/
- services/status-service/
- services/reservation-service/

```
- services/analytics-service/
- services/billing-service/

Final diagrams:
- docs/diagrams/final/README.md
- all .puml files under docs/diagrams/final/

Assignment/provider sources:
- docs/assignment/
- docs/assignment/providers/yaml/redPlug.yaml
- docs/assignment/providers/yaml/greenPlug.yaml
- docs/assignment/providers/yaml/bluePlug.yaml

GitHub:
- Inspect issues #1 through #14, especially #14.
- Confirm whether issues are closed/completed where expected.
- Confirm there are no fallback Markdown project-management artifacts such as docs/
- Confirm the GitHub Project state if visible through the connector; if not visible

What I want from you:

First, inspect the repo deeply and answer:

1. Is the implementation complete enough for the course submission?
2. Does the implementation match AGENTS.md and docs/specs/SPECIFICATIONS.md?
3. Are all services present and correctly separated by responsibility?
4. Does the frontend call only gateway paths?
5. Are provider services kept internal and not exposed directly to the browser?
6. Is RabbitMQ used only for background/event flows, not direct frontend actions?
7. Is the one-PostgreSQL-container-per-stateful-service rule preserved?
8. Are final docs and final PlantUML source diagrams present and coherent?
9. Are AI logs complete and consistent?
10. Are there any blockers before final submission?
11. Are there any non-blocking warnings that should be documented?
12. Is Prompt 14's completed_with_warnings status acceptable?

Then produce the final answer in this structure:

## Final repository inspection result

A concise but evidence-based conclusion.

## What is confirmed

Use a table with:
- Area
- Status
- Evidence / notes

## Warnings / non-blocking issues

Mention only real issues grounded in repo evidence.
```

```
## Blockers

Say “No blockers” only if the repo evidence supports it.

## Final Codex full verification prompt

Give me one exact copy-paste prompt for Codex to run a final verification-only pass

The Codex prompt must:
- not implement new features,
- not change backend/frontend business logic,
- not change architecture,
- not add deployment/Vercel,
- not create .env or secrets,
- rerun all available tests/builds/checks,
- verify Docker Compose startup and gateway health if Docker is available,
- verify authenticated EV/provider/operator smoke checks,
- verify frontend source has no forbidden direct calls,
- verify README/final docs/diagram presence,
- optionally validate PlantUML rendering only if plantuml is installed,
- update or add a final verification run report under docs/ai/codex-runs/,
- update docs/final-verification.md only if the verification results change,
- update docs/ai/prompt-log.md only if it creates a new verification run entry,
- clearly report any unavailable tools instead of claiming success,
- not create fallback project-management Markdown artifacts,
- not close or reopen issues unless explicitly instructed.

Important: The final Codex prompt should be a verification-only prompt, not an impl

Do not give me generic advice. Ground the answer in the repository inspection.
```

Πώς να το χρησιμοποιήσεις

Στη νέα συνομιλία:

1. Βάλε το παραπάνω prompt.
2. Σύνδεσε/δώσε τον GitHub connector για `ntua-el21026/NTUA-saasPlug`.
3. Ζήτα του να ξεκινήσει με repo inspection, όχι με Codex prompt απευθείας.
4. Η τελική απάντηση που θέλεις από εκείνη τη συνομιλία είναι:
   - επιβεβαίωση ότι το repo είναι έτοιμο,
   - warnings/blockers,
   - ένα **verification-only** prompt για Codex.